

Formal Methods and Specification (SS 2025/26)

Lecture 11: Automated Formal Software Verification

Stefan Ratschan

Czech Technical University in Prague



Evropský sociální fond Praha & EU: Investujeme do vaší budoucnosti

Leftover Problem

For proving correctness of loops, we need loop **invariants**.

Ideally:

- ▶ developed **before** program
- ▶ **documented** in the form of assertions

Reality is not ideal

Workarounds:

- ▶ Symbolic execution: check **individual** execution **paths**
- ▶ Bounded software model checking: check a certain **number of steps**

Synthesis of Loop Invariants

hardware design: finite state systems: viz MI-TES

software: active research area

this lecture: some basic techniques

both for humans and machines

Weakest Preconditions: Example

Spec:

Input:

Output: $z \geq 0$

Program:

$y \leftarrow x$

$x \leftarrow -10$

$z \leftarrow x + y$

Question: For which input is the program correct?

Weakest Preconditions

assume $I(x, y, z)$

$y \leftarrow x$

$x \leftarrow -10$

$z \leftarrow x + y$

@ $z \geq 0$

Question: For which condition I is the program correct?

A program is (*partially*) *correct* iff

every **regular** program **execution**

from the first program line to a program line with an @-assertion,
satisfies this **@-assertion**.

⊥

weaker condition? $x \geq 70$? (demo) weakest?

Weakest Preconditions

Example:

```
assume  $I(x, y, z)$   
 $y \leftarrow x$   
 $x \leftarrow -10$   
 $z \leftarrow x + y$   
 $@ z \geq 0$ 
```

A formula I is a *precondition* of a program P and a formula O iff

- ▶ every free variable of I is a program variable of P and
- ▶ the program **assume** I ; P ; $@O$ is correct

Every precondition I of a program P and a formula O is

- a *weakest precondition* of P and O iff
for every precondition I' of P and O , $\models I' \Rightarrow I$.

Computation of Weakest Preconditions: Example

$y \leftarrow x$

$x \leftarrow -10$

$z \leftarrow x + y$

@ $z \geq 0$

Verification condition:

$$[y_1 = x \wedge x_1 = -10 \wedge z_1 = x_1 + y_1] \Rightarrow z_1 \geq 0$$

Result:

$$\forall x_1, y_1, z_1 . [y_1 = x \wedge x_1 = -10 \wedge z_1 = x_1 + y_1] \Rightarrow z_1 \geq 0$$

Simplification:

$$\forall x_1, y_1 . [y_1 = x \wedge x_1 = -10] \Rightarrow x_1 + y_1 \geq 0$$

$$\forall y_1 . y_1 = x \Rightarrow -10 + y_1 \geq 0$$

$$-10 + x \geq 0, x \geq 10$$

Check:

assume $x \geq 10$; $y \leftarrow x$; $x \leftarrow -10$; $z \leftarrow x + y$; @ $z \geq 0$

Computation of Weakest Preconditions

Spec: Input: program P without control structures
 Output: O

Weakest precondition in quantified form:

$$\forall \vec{v} . VC(P; @O)$$

where $\vec{v}$ is the tuple of auxiliary variables of $VC(P; @O)$.

Here: VC must use the original variable names
for variables corresponding to initial values.

Problem: $\forall \vec{v}$

Quantifier Elimination:

- ▶ Given: formula ϕ
- ▶ Find: formula ϕ' s.t. $\models \phi \Leftrightarrow \phi'$, but ϕ' is quantifier free

Quantifier Elimination

for variables introduced by assignments: easy

for variables introduced by user input or procedure calls: not!

Example: **input** x
@ $x^2 + a \geq 1$

Weakest precondition:

$$\forall x . x^2 + a \geq 1$$

$$\forall x . x^2 \geq 1 - a$$

$$0 \geq 1 - a$$

$$a \geq 1$$

In general: very difficult problem

Computation of Strongest Postconditions: Example

assume $x \geq 10$

$y \leftarrow x$

$x \leftarrow -10$

$z \leftarrow x + y$

@ $O(x, y, z)$

should be correct.

Verification condition

$$[x \geq 10 \wedge y_1 = x \wedge x_1 = -10 \wedge z_1 = x_1 + y_1] \Rightarrow O(x_1, y_1, z_1)$$

should be valid.

Computation of Strongest Postconditions: Example

$$\models [x \geq 10 \wedge y_1 = x \wedge x_1 = -10 \wedge z_1 = x_1 + y_1] \Rightarrow O(x_1, y_1, z_1)$$

iff

$$\models \forall x . [x \geq 10 \wedge y_1 = x \wedge x_1 = -10 \wedge z_1 = x_1 + y_1] \Rightarrow O(x_1, y_1, z_1)$$

iff

$$\models [\exists x . x \geq 10 \wedge y_1 = x \wedge x_1 = -10 \wedge z_1 = x_1 + y_1] \Rightarrow O(x_1, y_1, z_1)$$

Result:

$$\exists x . x \geq 10 \wedge y_1 = x \wedge x_1 = -10 \wedge z_1 = x_1 + y_1$$

which is the symbolic execution condition with

the variables corresponding to non-final values quantified away:

$$\exists x . SE(\textbf{assume } x \geq 10; y \leftarrow x; x \leftarrow -10; z \leftarrow x + y)$$

Intuition: values at line of final assertion, reachable by program.

Computation of Strongest Postconditions

Spec: **Input:** I program P without control structures
 Output:

Strongest postcondition in quantified form, with indexed variables:

$$\exists \vec{v} . SE(\mathbf{assume} \ I; P)$$

where $\vec{v}$ is a tuple of the variables in $SE(\mathbf{assume} \ I; P)$
not corresponding to a final value.

In general, no simple way of elimination quantifiers (demo).

In practice: clever approximation instead of exact quantifier elimination

To relate to program variables, rename variables.

Example: $\exists x' . x' \geq 10 \wedge y = x' \wedge x = -10 \wedge z = x + y$

Pre- or Postcondition Based Software Verification

Given verification problem:

- ▶ Spec: **Input:** I
 Output: O
- ▶ Program: P

Correctness Proof:

- ▶ weak enough precondition: I' s.t. $\models I \Rightarrow I'$
- ▶ strong enough postcondition: O' s.t. $\models O' \Rightarrow O$

similar for internal assertions

We already know how to compute pre- and postconditions for basic paths

What about programs with loops?

Postcondition Computation: Example

Specification:

- ▶ Input: array a
- ▶ Output: r s.t. $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$

Program:

```
 $r \leftarrow \perp$   
for  $i \leftarrow 1$  to  $n$  do  
    if  $a[i] = 7$  then  $r \leftarrow \top$   
 $@ r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$   
return  $r$ 
```

For ensuring correctness:

compute a postcondition that implies the final assertion

Postcondition Computation: Example

```
 $r \leftarrow \perp$   
for  $i \leftarrow 1$  to  $n$  do  
  @ ???  
  if  $a[i] = 7$  then  $r \leftarrow \top$   
  @ ???  
return  $r$ 
```

¹			⁴
4	3	7	6

i	r
1	$\perp$
2	$\perp$
3	$\perp$
4	$\top$

Invariant involved in three verification conditions:

- ▶ holds in **first** loop iteration
- ▶ if it holds, and the loop is **re-entered**, then it must hold **again**
- ▶ if it holds, and the loop is **left**, then assertion **after** the loop must hold

Postcondition Computation: Example

```
 $r \leftarrow \perp$   
for  $i \leftarrow 1$  to  $n$  do  
  @  $\perp$   
  if  $a[i] = 7$  then  $r \leftarrow \top$   
@  $\perp$   
return  $r$ 
```

Basic Path:

```
 $r \leftarrow \perp$   
assume  $i = 1$   
@  $i = 1 \wedge \neg r$ 
```

incorrect

to make correct: strongest postcondition

Postcondition Computation: Example

```
 $r \leftarrow \perp$   
for  $i \leftarrow 1$  to  $n$  do  
  @  $i = 1 \wedge \neg r$   
  if  $a[i] = 7$  then  $r \leftarrow \top$   
@  $\perp$   
return  $r$ 
```

Basic Paths (without unnecessary assumes):

```
assume  $i = 1 \wedge \neg r$   
assume  $a[i] = 7$   
 $r \leftarrow \top$   
 $i \leftarrow i + 1$   
@  $[i = 1 \wedge \neg r] \vee [a[1] = 7 \wedge r \wedge i = 2]$ 
```

```
assume  $i = 1 \wedge \neg r$   
assume  $a[i] \neq 7$   
 $i \leftarrow i + 1$   
@  $[i = 1 \wedge \neg r] \vee [a[1] \neq 7 \wedge \neg r \wedge i = 2]$ 
```

$$[i = 1 \wedge \neg r] \vee [a[1] = 7 \wedge r \wedge i = 2] \vee [i = 1 \wedge \neg r] \vee [a[1] \neq 7 \wedge \neg r \wedge i = 2]$$

$$[i = 1 \wedge \neg r] \vee [i = 2 \wedge [r \Leftrightarrow a[1] = 7]]$$

Postcondition Computation: Example

```
 $r \leftarrow \perp$   
for  $i \leftarrow 1$  to  $n$  do  
  @  $[i = 1 \wedge \neg r] \vee [i = 2 \wedge [r \Leftrightarrow a[1] = 7]]$   
  if  $a[i] = 7$  then  $r \leftarrow \top$   
@  $\perp$   
return  $r$ 
```

Basic Paths:

```
assume  $i = 2 \wedge [r \Leftrightarrow a[1] = 7]$   
assume  $a[i] = 7$   
 $r \leftarrow \top$   
 $i \leftarrow i + 1$   
@  $[i = 1 \wedge \neg r] \vee [i = 2 \wedge [r \Leftrightarrow a[1] = 7]]$ 
```

```
assume  $i = 2 \wedge [r \Leftrightarrow a[1] = 7]$   
assume  $a[i] \neq 7$   
 $i \leftarrow i + 1$   
@  $[i = 1 \wedge \neg r] \vee [i = 2 \wedge [r \Leftrightarrow a[1] = 7]]$ 
```

$$[i = 1 \wedge \neg r] \vee [i = 2 \wedge [r \Leftrightarrow a[1] = 7]] \vee [i = 3 \wedge r \Leftrightarrow [a[1] = 7 \vee a[2] = 7]]$$

Postcondition Computation: Generalization

```
 $r \leftarrow \perp$   
for  $i \leftarrow 1$  to  $n$  do  
     $[i = 1 \wedge \neg r] \vee$   
    @  $[i = 2 \wedge r \Leftrightarrow a[1] = 7] \vee$   
     $[i = 3 \wedge r \Leftrightarrow [a[1] = 7 \vee a[2] = 7]]$   
    if  $a[i] = 7$  then  $r \leftarrow \top$   
@  $\perp$   
return  $r$ 
```

How to finish the process?

$\top$? results in a correct postcondition, but

The resulting postcondition must imply $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$

Generalized loop invariant:

$$r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$$

Checking the Invariant

$r \leftarrow \perp$

for $i \leftarrow 1$ **to** n **do**

$@ r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$

if $a[i] = 7$ **then** $r \leftarrow \top$

$@ \perp$

return r

¹			⁴
4	3	7	6

i	r
1	$\perp$
2	$\perp$
3	$\perp$
4	$\top$

Paths through the loop:

assume $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$

assume $a[i] = 7$

$r \leftarrow \top$

$i \leftarrow i + 1$

assume $i \leq n$

$@ r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$

assume $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$

assume $a[i] \neq 7$

$i \leftarrow i + 1$

assume $i \leq n$

$@ r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$

Resulting Postcondition

```
 $r \leftarrow \perp$   
for  $i \leftarrow 1$  to  $n$  do  
     $@ r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$   
    if  $a[i] = 7$  then  $r \leftarrow \top$   
 $@ r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$   
return  $r$ 
```

Postcondition Computation of Program with Loops

Input: program P s.t.

- ▶ every loop has an assertion (i.e., an invariant)
- ▶ the last line is an assertion
- ▶ all assertions are of the form $@ \perp$

Output: program P' s.t.

- ▶ P' is identical to P except for formulas at assertions
- ▶ the formula of the final assertion is a postcondition of the program
(hopefully strong enough)

postcond(P)=

if all basic paths of P are correct **then** P

else postcond(P_w), where P_w is equal to P except for
some assertions $@ \alpha$ replaced by $@ \text{*simplify*}(\alpha \vee \pi)$ where
 π is a postcondition of a basic path ending with $@ \alpha$

+ generalization to ensure convergence

Precondition Computation

assume $\top$

$r \leftarrow \perp$

for $i \leftarrow 1$ **to** n **do**

if $a[i] = 7$ **then** $r \leftarrow \top$

$@ \top$

$@ r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$

return r

again has incorrect basic paths

Start with the weakest precondition,

ensuring that the basic path leaving the loop is correct

assume $i = n$

$@ r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$

Verification condition:

$$i = n \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]]$$

Precondition Computation

```
 $r \leftarrow \perp$   
for  $i \leftarrow 1$  to  $n$  do  
  if  $a[i] = 7$  then  $r \leftarrow \top$   
  @  $i = n \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]]$   
@  $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$   
return  $r$ 
```

Basic paths in loop, again ignoring initial assume:

```
 $i \leftarrow i + 1$   
assume  $i \leq n$   
assume  $a[i] = 7$   
 $r \leftarrow \top$   
@  $i = n \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]]$ 
```

```
 $i \leftarrow i + 1$   
assume  $i \leq n$   
assume  $a[i] \neq 7$   
@  $i = n \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]]$ 
```

Weakest Precondition: **if**

$i \leftarrow i + 1$

assume $i \leq n$

assume $a[i] = 7$

$r \leftarrow \top$

@ $i = n \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]]$

already correct (both sides of equivalence obviously true)

Intuition: after execution of the **if** branch,
the result is correct, independently of the initial state

Weakest precondition: $\top$

Weakest Precondition: **else**

$i \leftarrow i + 1$

assume $i \leq n$

assume $a[i] \neq 7$

@ $i = n \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]]$

$$\forall i_1 . \left[\begin{array}{l} [i_1 = i + 1 \wedge i_1 \leq n \wedge a[i_1] \neq 7 \wedge i_1 = n] \Rightarrow \\ r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7] \end{array} \right]$$

QE:

$$[a[n] \neq 7 \wedge i + 1 = n] \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]]$$

Backward Computation: Merging of Branches

assume $\top \wedge [a[n] \neq 7 \wedge i + 1 = n] \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]]$

$i \leftarrow i + 1$

if $a[i] = 7$ **then** $r \leftarrow \top$

@ $i = n \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]]$

In loop:

$r \leftarrow \perp$

for $i \leftarrow 1$ **to** n **do**

if $a[i] = 7$ **then** $r \leftarrow \top$

 @ $i = n \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]] \wedge$
 $[a[n] \neq 7 \wedge i + 1 = n] \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]]$

@ $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$

return r

Backward Computation: Generalization

$r \leftarrow \perp$

for $i \leftarrow 1$ **to** n **do**

if $a[i] = 7$ **then** $r \leftarrow \top$

 @ $i = n \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]] \wedge$
 $[a[n] \neq 7 \wedge i + 1 = n] \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]]$

@ $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$

return r

Again we need to finish the process

$$[\forall k \in \{i + 1, \dots, n\} . a[k] \neq 7] \Rightarrow [r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]]$$

Also holds initially, so this is another, **different loop invariant!**

Precondition Computation of Program with Loops

Input: program P s.t.

- ▶ every loop has an assertion
- ▶ the first line is an **assume**-assertion
- ▶ all assertions are of the form $@ \top$

Output: program P' s.t.

- ▶ P' is identical to P except for formulas at assertions
- ▶ the formula of the first assertion is a precondition of the program
(hopefully weak enough)

$\text{precond}(P) =$

if all basic paths of P are correct **then** P

else $\text{precond}(P_s)$, where P_s is equal to P except for

some assertions $@ \alpha$ replaced by $@ \text{simplify}(\alpha \wedge \pi)$ where

π is a precondition of a basic path starting with $@ \alpha$

+ generalization to ensure convergence

Summary

Computation of postconditions:

- ▶ **forward** computation
- ▶ initialize assertions with $\perp$
- ▶ iteratively **weaken**

Computation of preconditions:

- ▶ **backward** computation
- ▶ initialize assertions with $\top$
- ▶ iteratively **strengthen**

Convergence not guaranteed

Our example: strongest conditions+generalization

Current Situation

active research topic

Software verification competition:

<https://sv-comp.sosy-lab.org/>

Usage in Industry:

- ▶ <http://www.absint.com/astree/>
- ▶ Mathworks Polyspace
- ▶ <https://www.imandra.ai>

Loop Invariant Positions

each loop:

- ▶ at least one assertion
- ▶ at any position

which to choose?

Typical Position

```
 $r \leftarrow \perp$   
for  $i \leftarrow 1$  to  $n$  do  
   $\textcircled{r} \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$   
  if  $a[i] = 7$  then  $r \leftarrow \top$   
 $\textcircled{r} \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$   
return  $r$ 
```

Basic paths re-entering the loop, and leaving the loop
both check the loop body.

avoid this?

Loop Invariants

$r \leftarrow \perp$

for $i \leftarrow 1$ **to** n **do**

$@ r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$

if $a[i] = 7$ **then** $r \leftarrow \top$

$@ r \Leftrightarrow [\exists k . 1 \leq k \leq i \wedge a[k] = 7]$

$@ r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 7]$

return r

Basic paths starting before the loop: same as before

Basic paths through the loop:

assume $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$

assume $a[i] = 7$

$r \leftarrow \top$

$@ r \Leftrightarrow [\exists k . 1 \leq k \leq i \wedge a[k] = 7]$

assume $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 7]$

assume $a[i] \neq 7$

$@ r \Leftrightarrow [\exists k . 1 \leq k \leq i \wedge a[k] = 7]$

simpler, rest?

Loop Invariants: **for**-loops

for $i \leftarrow l$ **to** u **do**

$@ P$

 ...

$@ P[i \leftarrow i + 1]$

$@ P[i \leftarrow u + 1]$

Basic path reentering loop:

assume $P[i \leftarrow i + 1]$

$i \leftarrow i + 1$

assume $i \leq u$

$@ P$

Shortcut:

for $i \leftarrow l$ **to** u **do**

$@@ P$

 ...

Basic path leaving loop:

assume $P[i \leftarrow i + 1]$

assume $i = u$

$@ P[i \leftarrow u + 1]$

Both hold automatically!

Loop Invariants: **while**-loops

while C **do**

$@ P$

 ...

$@ P$

$@ P$

Shortcut:

while C **do**

$@@ P$

 ...

Basic path reentering loop:

assume P

assume C

$@ P$

Basic path leaving loop:

assume P

assume $\neg C$

$@ P$

again hold automatically!

Conclusion

Automated formal software verification is **difficult**

Industrial use in **specific applications**

Mostly for **simple properties** (no division by zero etc.)

Huge progress each year

Aaron Bradley and Zohar Manna. *The Calculus of Computation*. Springer, 2007.